Project Report

Wildlife Information System (WIS)

Document Number: WIS-SRS-001
Version: 3.0
Date: June 2026

Prepared by:
Muhammad Ehtisham
Fahad Khan
Muhammad Abbas

4. DESIGN

This chapter translates the WIS requirements and analysis models into a logical software design. It follows the technology choices in the SRS: HTML/CSS/JavaScript for the basic web interface, Node.js with Express.js for the backend, PostgreSQL for persistence, REST/JSON for API communication, and JWT for authentication.

4.1 MVC Architecture Design

WIS is organized using the Model–View–Controller (MVC) pattern within its three-tier structure. The View presents the web interface and API-facing output. The Controller receives requests, applies request validation and authorization, and coordinates operations. The Model represents the wildlife and user data and handles data access to PostgreSQL.

The MVC responsibilities are a logical organization of the application layer, not a change to the SRS technology stack.

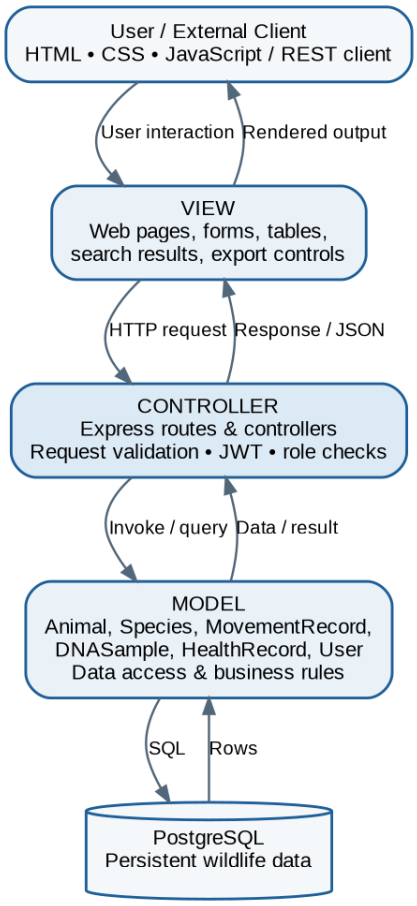


Figure 4.1: Logical MVC architecture within the WIS three-tier system.

Layer responsibilities

View: login, dashboard, animal list and forms, movement records, API documentation, and user-management screens.

Controller: Express routes/controllers, input validation, JWT authentication and role checks, request orchestration, and HTTP response formatting.

Model: Animal, Species, MovementRecord, DNASample, HealthRecord, and User domain objects plus persistence operations.

Database: PostgreSQL stores wildlife records, user information, and related data.

4.2 Class Diagram

The class diagram captures the principal classes and operations identified in the SRS. Attributes and methods follow the class descriptions in the source document. UUID and common data types are used as illustrative design-level type annotations; the SRS does not prescribe physical SQL column types.

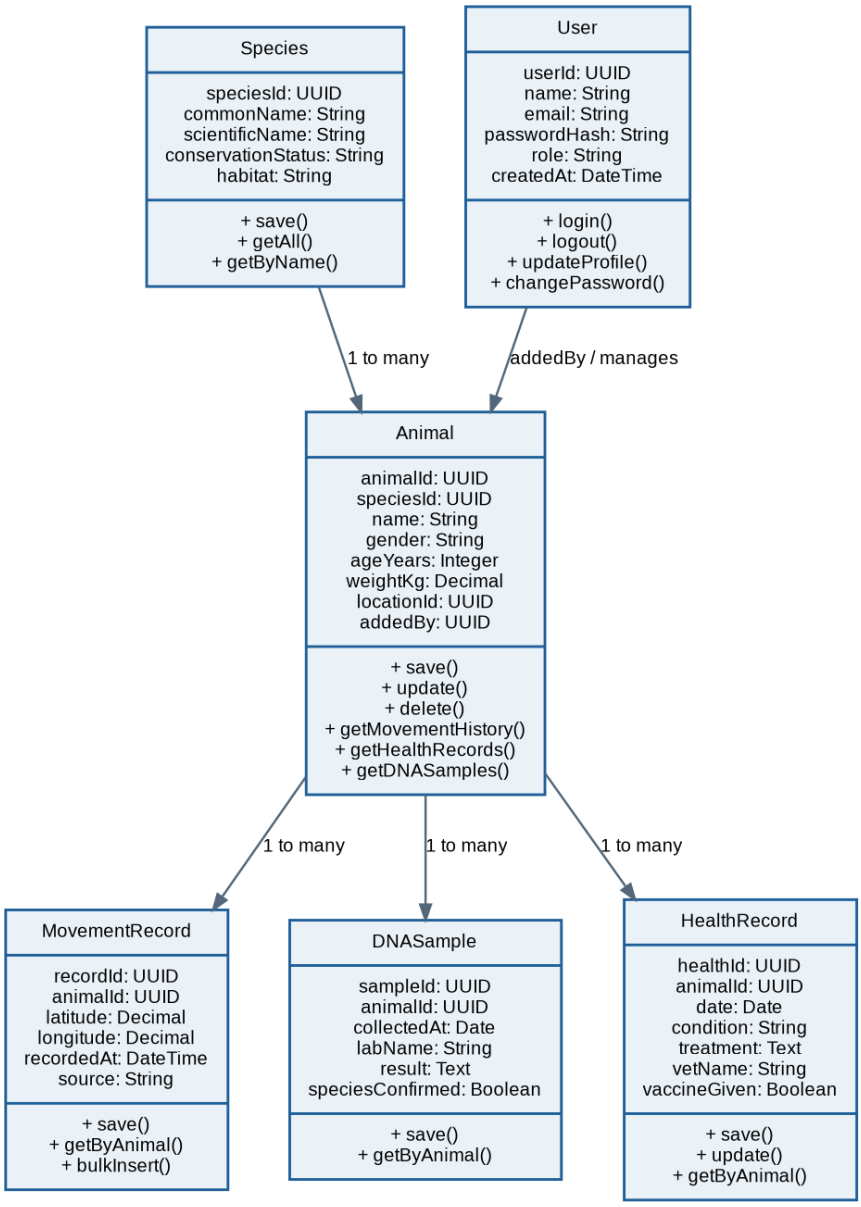


Figure 4.2: WIS class diagram showing key attributes, operations, and associations.

Design notes

Animal provides access to its movement history, health records, and DNA samples. Species supplies classification details. MovementRecord supports individual saving, retrieval by animal, and bulk insertion for CSV imports. DNASample and HealthRecord maintain biological and veterinary information linked to an animal. User supports authentication and account-related operations. The diagram is a logical class view and does not imply that every listed operation must be implemented as a public API endpoint.

4.3 Interface Design

The interface is a basic English-language web application. The SRS specifies a minimal blue-and-white visual theme, descriptive button labels, red error messages, and green success messages. The follow ing interface map shows the principal screens and navigation relationships; it is a structural design diagram rather tha n a pixel-perfect mock-up.

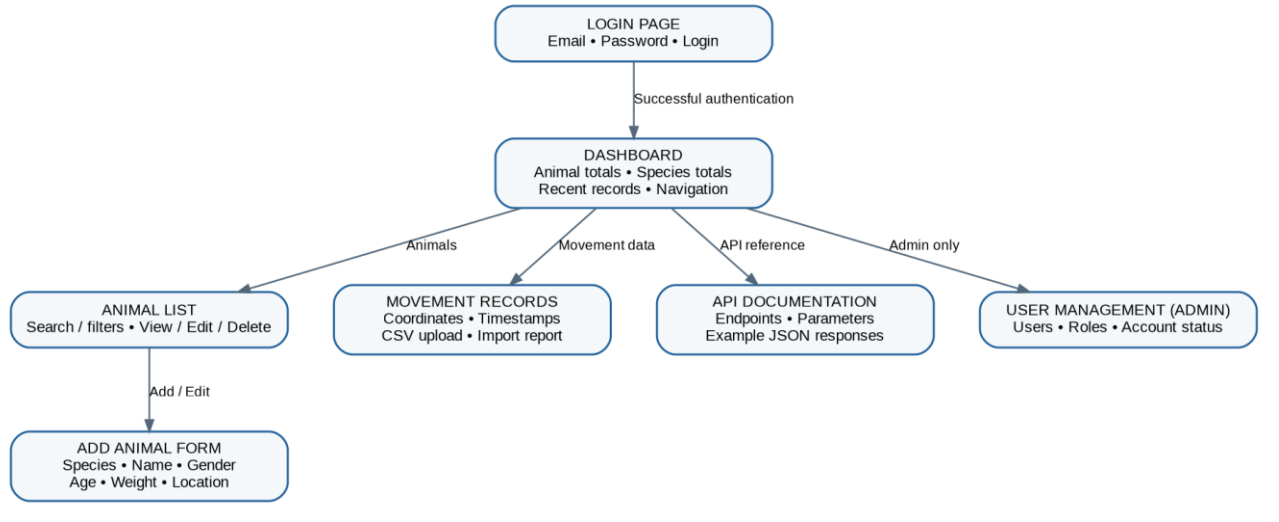


Figure 4.3: Interface navigation map for the main WIS web screens.

Screen descriptions

Screen	Main elements / purpose
Login Page	Email and password fields, Login button, and an error message for invalid credentials.
Dashboard	Welcome message, total animal count, number of species, recent records, and navigation.
Animal List	Sortable animal table with search/filter and View, Edit, and Delete actions according to role.
Add Animal Form	Species, name, gender, age, weight, and location fields with a submit action.
Movement Records	GPS coordinates and timestamps for an a nimal; CSV upload and import result summary.
API Documentation	Available endpoints, required parameters, and example JSON responses.
User Management	Admin-only user list and controls for creating or removing accounts / managing roles.

Access should follow the role permissions defined in the SRS: administrators have full access; data entry operators have read/write access; researchers can read and export; government officials have read-only access; and developers use authenticated API access.

4.4 Sequence Interaction Diagram

This interaction diagram shows how the API query request passes through the MVC components. It expands the system-level sequence from Chapter 3 and illustrates the responsibilities of the Express route, JWT middleware, controller, model/service, and PostgreSQL database.

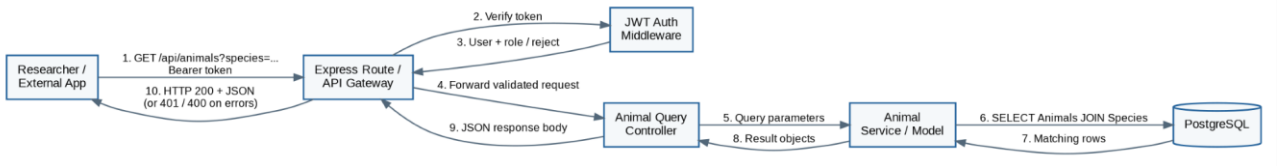


Figure 4.4: Detailed sequence interaction for an animal search API request.

Interaction steps

1. The researcher or external application sends a GET request with a species filter and Bearer token.
2. The Express route forwards the request to JWT authentication middleware.
3. Middleware verifies the token and obtains the user identity/role; invalid credentials are rejected.
4. The route passes an authorized request to the Animal Query Controller.
5. The controller forwards validated query parameters to the Animal Service/Model.
6. The model executes a PostgreSQL query joining animal and species information as required.
7. PostgreSQL returns matching ro ws to the model.
8. The model returns result objects to the controller, which prepares the JSON response.
9. Express returns HTTP 200 with the JSON payload. Invalid token or invalid parameters produce the corresponding TTP 401 or HTTP 400 response.

4.5 Design Constraints and Traceability

The design remains within the scope of the SRS. WIS stores movement information received from external tracking devices but does not design or install GPS collars, provide live animated tracking maps, or perform AI/ML identification. It provides a web interface and RES T API, uses PostgreSQL for structured storage, and does not introduce mobile applications or automated outbreak alerts. The principal design elements trace to the SRS functional requirements for animal profiles, movement data, DNA and health records, API access, user management, search/filter, CSV upload, and export.

Source alignment note: This document reorganizes and explains the Chapter 3 and Chapter 4 material in original wording. Where the SRS does not define a detail ed schema or implementation behavior, the diagrams keep the representation conceptual rather than asserting an unprovided technical detail.